Coding Standards

Coding standards are enforced using mainly Sonarqube, check on each pull request, and Black, part of local and PR quality checks.

Sonarqube

Names of variables and function are in lower-case snake-case:

i.e. example_variable, example_function_name()

For clarity, all function names must be self explanatory or have comments above the function.

The amount of duplicate code across the whole repo must be less than 3% of total code. This is achieved by the use of shared helper functions wherever possible.

Code complexity is capped at 15 by default. This complexity score is a measure of how hard it is for humans to understand the code, and is measured for each function/module. Sonarqube applies penalty scores for complex elements like flow control structures, nested statements, and logical operators. These elements are kept to a minimum and only used if necessary.

Enforcing a low complexity on code reduces bug rates, simplifies onboarding, and reduces maintenance costs.

Sonarqube also gives code a reliability rating based on the number of potential reliability issues on both new and existing code, as well as the amount of effort to resolve these issues. A raring of A is required, meaning 0 or more info issues, no low or higher issues

Black

Black is a package library used to standardize and enforce format of python files in the backend folder of the repo.

Some of the formatting rules it enforces include:

- Only one full expression or simple statement per line
- Line lengths are limited to 88 characters
- Maximum one line of whitespace between code within functions
- Double lines of whitespace allowed between function/modules
- Enforces use of double quotes
- Lowercase character literals
- Trailing commas in lists and other structures
- Consistent indentation and bracket positioning

Repository File Structures

- .github
  - Docs #Documentation
  - instructions #Sonarqube calibration
  - workflows #.yml files and scripts for CI/CD and deployment pipelines
- backend
  - app
    - api #API routes
    - services #Business Logic
    - models #SQLModel schemas
    - tests #Unit and integration tests
    - utils #Logging, helpers
    - main.py #App entry point
  - requirements-dev.txt
  - README.md
- frontend
  - public #Static files (favicon, manifest)
  - src
    - tests #Vitest tests
    - api #Frontend API connections
    - assets #Images, fonts (bundled)
    - components #React components
    - landing/imports/Landing #Landing page
    - lib #Frontend basic data processing and logic
    - pages #Page components
    - styles #Theme, fonts, globals, etc.
    - types #Variable type interfaces
    - App.tsx #Entry component
    - Routes.tsx #Website routing logic
    - main.tsx #Rendering logic for App
  - README.md